

UNITÉ 5 · SEMAINE 7

Tubes et composition de processus

📅 Semaine 7 · dates à confirmer </> Laboratoire TD4 (corrigé + défi IA) ~ 2 animations

MANUEL M5 · THÉORIE DE L'UNITÉ

Tubes et composition de processus

Lecture de base de l'unité 5 (≈ 1 h). Les sorties sont réelles: programmes du TD4 exécutés sur la machine du cours.

SOMMAIRE

- | | | | |
|----|--|----|--|
| §1 | Le tube: un tampon à deux extrémités | §3 | Rediriger avec <code>dup2</code> et exécuter |
| §2 | Les règles: blocage, fin de fichier et SIGPIPE | §4 | Pipelines de N commandes |
| | | §5 | Au-delà |

§1 Le tube: un tampon à deux extrémités

`pipe(fds)` crée un tampon circulaire dans le noyau et renvoie deux descripteurs: `fds[0]`, l'extrémité de **lecture**, et `fds[1]`, l'extrémité d'**écriture**. Ce qui est écrit dans l'une se lit dans l'autre, dans l'ordre, comme un flux d'octets sans frontières de messages. Le tube n'a pas de nom (contrairement à une FIFO, unité 3): seuls les processus qui héritent de ses descripteurs peuvent l'utiliser, c'est pourquoi on le crée **avant** `fork`. Ensuite, chaque processus **ferme l'extrémité qu'il n'utilise pas**: le parent qui écrit ferme `fds[0]`, l'enfant qui lit ferme `fds[1]`.

C tds/td4/mypipe.c

```
/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a child
 (reader).
 * ./mypipe [count]      the parent writes "Hello World\n" `count` times (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the pipe
 capacity)
 * the parent blocks on write() once the pipe is full. When the child exits, nobody can
 read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken pipe", status
 141
 * in the shell). That only works if the parent CLOSED its own read end: with
 * -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a potential
```

```

reader,
    * the kernel never raises SIGPIPE and the parent blocks forever.
    */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE    "Hello World\n"
#define READ_MAX   500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end, fds[1] write
end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                            /* child: reader */
        exit_if(close(fds[1]) == -1, "close"); /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
        exit_if(n == -1, "read");
        exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
        fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
        exit_if(close(fds[0]) == -1, "close");
        return EXIT_SUCCESS;
    }

    /* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");      /* THE line that exercise 2 is
about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n", count);
    exit_if(close(fds[1]) == -1, "close");
    exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");

```

```
return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave
```

Pratiquez → TD4 · 1

§2 Les règles: blocage, fin de fichier et SIGPIPE

- **Lire** dans un tube vide bloque jusqu'à l'arrivée de données... sauf s'il ne reste **aucun** descripteur d'écriture ouvert dans aucun processus: alors `read` renvoie 0, fin de fichier.
- **Écrire** dans un tube plein bloque jusqu'à ce que quelqu'un lise. La capacité par défaut sous Linux est de 64 Kio (65 536 octets); on la consulte et la modifie avec `fcntl(F_GETPIPE_SZ / F_SETPIPE_SZ)`.
- Écrire alors qu'il ne reste **aucun lecteur** déclenche le signal `SIGPIPE`, qui tue le processus par défaut (statut 141 = 128 + 13 dans le shell); s'il est ignoré, `write` échoue avec `EPIPE`.
- Une écriture d'au plus `PIPE_BUF` = 4096 octets est **atomique**: elle ne s'entrelace pas avec celles des autres écrivains.

Le TD4 · 2 montre les trois premières règles d'un coup. L'enfant lit 500 octets une fois et se termine; le parent écrit N messages de 12 octets. Si le parent finit avant que l'enfant parte, tout va bien; si l'enfant part d'abord, le `write` suivant du parent reçoit `SIGPIPE`. Avec beaucoup de messages, cela arrive toujours, mais entre les deux, c'est une **course** entre deux processus qui s'exécutent sur des cœurs différents: le résultat change d'une exécution à l'autre. Mesuré 30 fois par taille:

Sortie réelle

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses; done
 50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
100 messages: 30 x status 0      0 x status 141 (SIGPIPE)
200 messages: 28 x status 0      2 x status 141 (SIGPIPE)
300 messages: 19 x status 0     11 x status 141 (SIGPIPE)
400 messages:  5 x status 0     25 x status 141 (SIGPIPE)
500 messages:  5 x status 0     25 x status 141 (SIGPIPE)
1000 messages: 0 x status 0     30 x status 141 (SIGPIPE)
```

Si le parent oublie de fermer `fds[0]`, il compte lui-même comme lecteur: il ne reçoit jamais `SIGPIPE`, remplit les 64 Kio et bloque pour toujours en attendant que « quelqu'un » lise:

Sortie réelle

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by
timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

Pratiquez → A5.2 capacité et course · TD4 · 2

§3 Rediriger avec `dup2` et exécuter

`dup2(fds[1], STDOUT_FILENO)` fait pointer le descripteur 1 vers le tube (unité 3: il copie le pointeur vers l'entrée de la table des fichiers ouverts). Comme `exec` conserve les descripteurs (unité 4), le programme exécuté ensuite écrira dans le tube sans le savoir. C'est ainsi qu'on relie les programmes des autres, comme `ps`, à un lecteur à nous:

C tds/td4/upper.c

```
int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                /* child */
        exit_if(close(fds[1]) == -1, "close"); /* otherwise it never sees end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }

    /* parent */
    exit_if(close(fds[0]) == -1, "close");
    exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
    exit_if(close(fds[1]) == -1, "close"); /* 1 already refers to the pipe */
    execlp("ps", "ps", (char *)NULL);      /* "ps" twice: file to run, then
    argv[0] */
    exit_if(1, "execlp ps");
    return EXIT_FAILURE;
}
```

Sortie réelle

```
$ ./upper | head -4
PID TTY          TIME CMD
6706 ?           00:00:25 UVICORN
9771 ?           00:00:03 SYSTEMD
9801 ?           00:00:00 (SD-PAM)
```

Pratiquez → TD4 · 3

§4 Pipelines de N commandes

Pour `c1 | c2 | ... | cN`, le shell crée $N - 1$ tubes et N enfants. L'algorithme tient dans une boucle avec une variable `in` qui garde l'extrémité de lecture du tube précédent: pour chaque commande, si ce n'est pas la dernière, `pipe` ; `fork` ; l'enfant fait `dup2(in, 0)` et `dup2(fds[1], 1)`, ferme tout le reste et appelle `exec` ; le parent ferme `in` et `fds[1]` et garde

`fds[0]` comme nouveau `in`. À la fin, il les attend tous et renvoie le statut de la **dernière** (dans `bash`, `set -o pipefail` renvoie celui de la dernière qui a échoué).

La règle d'or: **chaque tube doit finir avec exactement un descripteur d'écriture et un descripteur de lecture**, dans les deux processus qui l'utilisent. Un seul `close` oublié — dans le parent ou dans un enfant qui a hérité du tube — laisse un écrivain en trop, et la dernière commande attend une fin de fichier qui n'arrive jamais.

C tds/td4/pipeline_args.c

```
int in = STDIN_FILENO;
pid_t last = -1;
for (int i = 0; i < ncommands; i++) {
    int fds[2] = { -1, -1 };
    if (i < ncommands - 1)
        exit_if(pipe(fds) == -1, "pipe");

    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        if (in != STDIN_FILENO) {
            exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
            exit_if(close(in) == -1, "close");
        }
        if (fds[1] != -1) {
            exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
            exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}
```

Sortie réelle

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Sortie réelle

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Pratiquez → A5.1 construire un pipeline · TD4 · 4 et 5

§5 Au-delà

- `pipe2(fds, 0_CLOEXEC)` crée le tube avec les descripteurs marqués pour être fermés à l'`exec` : cela évite qu'un programme lancé hérite d'extrémités qui ne lui appartiennent pas (et qui, comme au §4, empêcheraient une fin de fichier).
- `splice` et `tee` déplacent des données entre un tube et un fichier ou un socket dans le noyau, sans les copier dans l'espace du processus.
- La **substitution de processus** de bash (`diff <(ls a) <(ls b)`) : le shell crée des tubes et les passe comme chemins `/dev/fd/N`.
- Une communication **bidirectionnelle** demande deux tubes ou un `socketpair` (unité 8).